ISIS4426 — Desarrollo de Soluciones Cloud

Entrega 3 — Aplicación elástica IaaS

Proyecto: Plataforma web de cursos masivos abiertos en línea (MOOC)
Periodo académico: 2026-20
Escalamiento automático de API y workers y consolidación del frontend funcional

Contexto

En la entrega 2, la plataforma MOOC se desplegó sobre dos máquinas virtuales de capacidad fija: una para la API y otra para los workers y la cola de procesamiento. La solución incorporó PostgreSQL administrado y almacenamiento de objetos del proveedor, y el equipo caracterizó su comportamiento mediante pruebas de actividad académica y procesamiento multimedia.

En esta entrega, la organización necesita responder a variaciones en la demanda sin aumentar o reducir manualmente el número de servidores de la API y de procesamiento asíncrono. El equipo deberá evolucionar ese despliegue hacia una arquitectura con balanceo de carga y escalamiento horizontal automático, asegurar que las instancias puedan reemplazarse sin perder estado de negocio y medir los efectos de este cambio frente a la línea base anterior.

El trabajo comprende cinco frentes: arquitectura elástica de API y workers sobre máquinas virtuales, finalización del frontend mínimo funcional, pruebas de carga comparativas, mejora de la calidad del código y actualización de la documentación de arquitectura y creación de la primera línea base de deuda técnica. Se evaluará la correspondencia entre las decisiones, la implementación y los resultados observados.

La entrega se desarrollará durante **cuatro semanas**, en los equipos conformados para el proyecto y de acuerdo con el cronograma del curso. Cada integrante deberá participar en la implementación, las pruebas, el análisis y la sustentación de la solución.

Objetivos

- **Diseñar y desplegar** grupos independientes de máquinas virtuales para API y workers que ajusten automáticamente su tamaño según políticas de escalamiento explícitas.
- **Distribuir** las solicitudes mediante un balanceador externo e incorporar nuevas instancias de API y nuevos consumidores de la cola sin intervención manual.
- **Completar** los flujos mínimos de frontend para que estudiantes y profesores utilicen la plataforma desplegada desde el navegador.
- **Separar** el estado persistente de las instancias de aplicación y conservar la integridad de sesiones, operaciones académicas y procesos asíncronos.
- **Comparar** el desempeño con la entrega 2 y explicar los beneficios, costos y límites del escalamiento implementado.
- **Evaluar** la calidad del código mediante análisis automatizado, atender los hallazgos prioritarios y establecer la primera línea base de deuda técnica con un plan de tratamiento.

Esquema de evaluación

La entrega corresponde al **10% de la nota final**, según el programa. Se evaluarán los siguientes criterios; esta tabla no introduce una distribución porcentual interna adicional.

Componente	Evidencia de cumplimiento
Arquitectura elástica	Dos grupos de 1 a 3 instancias, creación y reducción automáticas en API y workers, arranque autónomo, balanceo de API y consumo compartido de trabajos
Frontend funcional	Recorrido completo de autoría y publicación por un profesor y de inscripción, consumo y evaluación por un estudiante, desde el navegador sobre el entorno cloud
Integridad y operación	Sesiones y reglas de negocio correctas entre réplicas, estado externo, recuperación segura de trabajos al retirar workers y controles de seguridad previos
Pruebas comparativas	Reproducción de los dos escenarios anteriores, ciclos de autoscaling en ambas capas, métricas verificables y análisis causal de resultados
Calidad del código	Primer análisis automático, verificación de umbrales, reporte trazable al commit final e interpretación de los resultados
Documentación y deuda	Diagramas del despliegue real, decisiones justificadas, instrucciones reproducibles y primera línea base de deuda con inventario priorizado y plan de tratamiento
Sustentación	Demostración verificable y capacidad de los integrantes para explicar configuración, decisiones y resultados

No satisface el componente de arquitectura un grupo ampliado únicamente de forma manual ni una solución que escale la API pero mantenga fija la capa worker. No satisface el componente de frontend una demostración que requiera sustituir pasos de los flujos mínimos por llamadas manuales a la API. Tampoco satisface el análisis de capacidad un informe sin línea base explícita o sin observación del autoscaling. Alcanzar umbrales de calidad sin documentar el diagnóstico inicial, los hallazgos pendientes y su plan de tratamiento no cumple plenamente el componente de calidad y deuda técnica.

Recomendaciones y consideraciones

Continuidad con la entrega anterior

Se conserva la plataforma implementada previamente: backend en Go como monolito modular, contenedores Docker, workers independientes, cola de procesamiento, PostgreSQL administrado y almacenamiento de objetos. Se mantienen las reglas de negocio y los flujos funcionales del alcance de las entregas anteriores.

Es obligatorio finalizar los pendientes del frontend necesarios para disponer de una aplicación funcional para estudiantes y profesores. El alcance mínimo se define más adelante; no se exige implementar todas las vistas de la especificación general ni ampliar el dominio del producto. Los recorridos funcionales deberán demostrarse desde el navegador, integrados con el backend desplegado. Los clientes HTTP, scripts y pruebas automatizadas complementan esa evidencia. La distribución de la aplicación entre varias máquinas virtuales no exige convertir sus módulos en microservicios.

El equipo conservará el proveedor de nube pública utilizado en la entrega 2. En GCP, la implementación de referencia utiliza **grupos de instancias administradas (MIG) independientes de Compute Engine para API y workers**, un balanceador externo, Cloud SQL y Cloud Storage. En otro proveedor, deberán utilizarse servicios equivalentes y documentar cómo satisfacen cada requisito. No se exige migrar de proveedor para esta entrega.

Cambios frente a la entrega anterior

Componente	Entrega 2	Entrega 3
API	Una máquina virtual de capacidad fija	Grupo de máquinas virtuales con escalamiento automático entre 1 y 3 instancias
Acceso a la API	Punto de entrada en Web Server	Balanceador externo con HTTPS y verificaciones de salud
Inicialización	Despliegue sobre una máquina previamente configurada	Inicialización automática y reproducible de cada nueva instancia
Workers	Worker Server con capacidad fija	Grupo independiente con escalamiento automático entre 1 y 3 instancias
Cola	Alojada junto con los workers	Servicio compartido fuera de las instancias que se crean y destruyen automáticamente
Frontend	Desarrollo no exigido en esa etapa	Flujos mínimos de estudiante y profesor completos e integrados con el backend cloud
Persistencia	PostgreSQL administrado y almacenamiento de objetos	Persistencia compartida por todas las instancias
Validación	Caracterización de capacidad	Comparación con la línea base y observación del ciclo de escalamiento
Calidad y deuda técnica	Sin línea base formal de deuda técnica	Primer diagnóstico, análisis automatizado, verificación de umbrales e inventario priorizado de deuda técnica

El **escalamiento automático es obligatorio tanto en la API como en la capa de workers**, con grupos, políticas y evidencias independientes. La cola será compartida y deberá alojarse fuera de ambos grupos elásticos, en una instancia dedicada o un servicio compatible con la solución. No debe crearse una cola independiente por réplica ni depender su disponibilidad de una máquina que el autoscaling pueda retirar.

No se exige alta disponibilidad multizonal, failover de base de datos, CDN, migración a Kubernetes, ejecución PaaS o serverless, ni despliegue continuo automatizado. Se conservan los controles de HTTPS, conectividad privada y acceso autorizado establecidos en la entrega 2. La infraestructura como código es recomendable; no se exige adoptar una herramienta nueva, pero sí entregar la configuración y el procedimiento necesarios para reproducir el entorno.

Frontend mínimo funcional

El resultado debe permitir completar los siguientes recorridos desde una interfaz web desplegada, sin recurrir a llamadas manuales a la API para suplir pasos faltantes. Se permite agrupar varias acciones en una misma vista y reutilizar los componentes existentes.

Rol	Flujos mínimos obligatorios
Estudiante	Iniciar y cerrar sesión; consultar el catálogo y el detalle de un curso publicado; inscribirse; acceder a sus cursos y navegar módulos, unidades y recursos; consumir contenido multimedia y documentos; consultar su progreso; presentar un quiz y consultar su resultado
Profesor	Iniciar y cerrar sesión; consultar sus cursos; crear y editar un curso propio con su estructura de módulos y unidades; agregar recursos, cargar archivos y consultar su estado de procesamiento; crear o editar un quiz básico; publicar el curso cuando cumpla las reglas de negocio y consultar el resultado de la publicación

Los flujos deben utilizar datos reales del entorno cloud y respetar las reglas de autorización, publicación, intentos, calificación y progreso ya definidas. La interfaz deberá mostrar estados de carga, contenido vacío, procesamiento, éxito y error cuando correspondan. Un archivo todavía en procesamiento no debe mostrarse como disponible.

No se exige un panel administrativo completo, analítica avanzada, un editor enriquecido con todas sus opciones ni todas las vistas del producto. Para la demostración se podrán preparar cuentas de estudiante y profesor mediante datos de prueba; no es obligatorio completar registro, recuperación de contraseña o administración de usuarios en la interfaz de esta entrega. Esto no exime de autenticar y autorizar las operaciones en el servidor.

Se deberá demostrar un recorrido conectado: el profesor crea y publica un curso con contenido procesado y un quiz; después, un estudiante lo encuentra, se inscribe, consume el contenido, registra progreso y presenta el quiz. Las acciones de ambos roles deben poder realizarse desde el navegador. Los datos

preparados pueden facilitar la demostración, pero no sustituir los pasos de autoría exigidos. La validación debe incluir que un estudiante no pueda ejecutar acciones reservadas al profesor y que un profesor no pueda modificar cursos ajenos.

Incluir una matriz breve de los flujos implementados, sus rutas de interfaz y la evidencia de verificación. No es suficiente entregar pantallas estáticas, datos simulados en el frontend o vistas sin conexión funcional con la API.

Modelo de despliegue elástico en la nube pública

Plantilla e inicialización de las instancias

Definir una plantilla de instancia o mecanismo equivalente para cada grupo, API y workers, que especifique:

- Tipo de máquina, capacidad de CPU, memoria y almacenamiento, e imagen del sistema operativo.
- Versión identificable de la imagen de contenedor y configuración necesaria para iniciar la API o los workers, incluidas las dependencias de procesamiento multimedia.
- Configuración de red, reglas de acceso e identidad de servicio.
- Mecanismo de inicialización automática: script de arranque, imagen de máquina personalizada o combinación de ambos.
- Obtención segura de secretos, conexión a los servicios compartidos y arranque de los contenedores.

Una máquina creada por cualquiera de los grupos deberá quedar lista para atender solicitudes o consumir trabajos de la cola, según su función, **sin acceso manual ni ajustes posteriores por parte del equipo**. El procedimiento debe manejar reinicios sin duplicar procesos o recursos. Las migraciones de base de datos deberán ejecutarse de forma controlada, evitando que cada réplica intente aplicarlas simultáneamente.

Se debe registrar el tiempo de arranque y explicar cómo se determina que la aplicación está lista. Las instancias de un mismo grupo y corrida deberán ejecutar la misma versión de código y configuración funcional. El frontend deberá desplegarse con una versión identificable; documentar su ubicación, su conexión con la API y la configuración de sesión y CORS cuando corresponda.

Grupos de instancias y políticas de escalamiento

Configurar **dos grupos independientes, cada uno con mínimo 1 y máximo 3 instancias**: uno de API y otro de workers. Los límites no incluyen el generador de carga, la infraestructura del frontend ni los servicios de cola y persistencia. Aumentar únicamente el número de procesos dentro de una máquina no acredita escalamiento horizontal de la capa worker.

Cada política deberá documentar:

Parámetro	Información requerida
Señal de escalamiento	API: CPU, solicitudes por instancia u otra señal justificada. Workers: profundidad o antigüedad de la cola, trabajos pendientes por consumidor u otra señal que represente la demanda de procesamiento

Parámetro	Información requerida
Objetivo o umbral	Valor configurado, unidad y justificación basada en el comportamiento de la aplicación
Inicialización	Tiempo previsto para que una nueva instancia pueda atender tráfico
Estabilización y reducción	Ventanas y controles que evitan oscilaciones o retiros prematuros, según el proveedor
Límites	Mínimo 1, máximo 3 y comportamiento al alcanzar el máximo

Documentar los parámetros efectivos del proveedor y su significado. La ampliación manual del grupo no acredita autoscaling. Durante las pruebas debe observarse, en cada grupo, al menos una creación automática de capacidad adicional y una reducción posterior al disminuir la demanda. En la API se verificará la incorporación al balanceador; en workers, el inicio efectivo del consumo y procesamiento de trabajos de la cola compartida.

Balanceador externo y verificaciones de salud

El balanceador será el punto público de entrada a la API mediante **HTTPS**. Configurar y documentar su dirección de acceso, certificado, listeners o reglas de reenvío, grupo de destinos o servicio de backend, puertos y política de distribución.

Las verificaciones de salud deberán especificar endpoint, protocolo, intervalo, timeout y umbrales de cambio de estado. El endpoint deberá reflejar la capacidad de la instancia para atender solicitudes, sin exponer secretos o información sensible.

Demostrar que el balanceador distribuye solicitudes entre al menos dos instancias saludables y deja de enviar nuevas solicitudes a una instancia que no supera las verificaciones. Para identificar los destinos se podrán utilizar logs correlacionados o un identificador técnico de instancia en las evidencias.

Documentar cómo se manejan las solicitudes en curso al retirar una instancia. Diferenciar la exclusión de un destino por el balanceador de su eventual reparación o reemplazo por el grupo de máquinas virtuales.

Persistencia y estado compartido

Las instancias de API y workers deberán compartir la base de datos PostgreSQL administrada y el almacenamiento de objetos, con los permisos que correspondan a su función. Los originales, derivados HLS, documentos, imágenes y demás archivos persistentes deberán permanecer en el servicio de objetos.

Se permite utilizar disco local para archivos temporales reconstruibles, con limpieza y manejo de interrupciones. Ninguna copia local podrá constituir la única fuente de datos necesarios para continuar una operación de negocio.

Las sesiones y los mecanismos de autorización deberán funcionar cuando solicitudes sucesivas lleguen a instancias diferentes. La afinidad de sesión, si se utiliza, no reemplaza el desacoplamiento del estado. Las cachés locales, si existen, deberán tener una política compatible con la consistencia requerida.

Los workers del grupo elástico consumirán la cola compartida. Mantener su persistencia y recuperación, así como los mecanismos de reintento e idempotencia existentes. Al reducir el grupo, dejar de aceptar nuevos trabajos en la instancia que se retira y permitir que los activos finalicen, o aplicar un mecanismo verificable de recuperación y reencolado seguro. Una cola sin trabajos pendientes puede tener trabajos en ejecución y no constituye por sí sola una señal suficiente para retirar consumidores. Los reintentos no deben producir derivados ni efectos de negocio duplicados. Documentar límites de concurrencia por instancia, manejo de archivos temporales y puntos únicos de falla que permanezcan en la cola y la base de datos.

Verificar, como mínimo:

- Continuidad de una sesión al alternar entre instancias de API.
- Lecturas y escrituras consistentes de cursos, inscripciones, intentos y progreso.
- Envío duplicado de un intento sin doble calificación ni efectos de negocio duplicados.
- Acceso a los archivos y al estado de los trabajos desde cualquier instancia autorizada.
- Conservación de datos de negocio al retirar o reemplazar una instancia de API.
- Finalización o recuperación verificable de trabajos al retirar un worker durante el procesamiento, sin pérdida de trabajos aceptados ni efectos de negocio duplicados.

Seguridad, operación y costos

Mantener PostgreSQL, la cola y los servicios de workers fuera del acceso público. Restringir el acceso a las instancias de API a los caminos necesarios para el balanceador, las verificaciones de salud y la administración autorizada.

Utilizar identidades de servicio con permisos mínimos y documentar por separado la autorización de acceso al servicio cloud y la autenticación de la aplicación ante PostgreSQL. Los secretos no deben incluirse en código, imágenes, plantillas, scripts versionados ni evidencias. Entregar ejemplos de configuración sin valores sensibles.

Registrar región, tamaños de instancia, costos estimados y consumo observado. La estimación debe contemplar ambos grupos con una y con tres instancias cada uno, así como frontend, balanceador, cola, base de datos, objetos, red y monitoreo. Indicar fecha, horas de uso y demás supuestos utilizados. Mantener alertas de presupuesto cuando la cuenta lo permita.

Después de conservar las evidencias y respaldos necesarios, eliminar los recursos de laboratorio que no deban seguir activos. Documentar su reconstrucción para la sustentación. Las pruebas evaluadas y la sustentación deberán utilizar PostgreSQL administrado; una base local de desarrollo no sustituye este requisito.

Documentación requerida

Arquitectura de la aplicación

Entregar un documento que explique la solución efectivamente desplegada y los cambios frente a la entrega anterior. Debe incluir:

- **Modelo de componentes:** frontend, API modular, workers, cola, base de datos y almacenamiento de objetos, con sus responsabilidades y comunicaciones síncronas y asíncronas.

- **Modelo de despliegue:** región, red, subredes, balanceador, grupos independientes de API y workers, contenedores, cola compartida, servicios de persistencia y reglas de acceso.
- **Decisiones de escalamiento:** plantillas, arranque automático, políticas de ambos grupos, verificaciones de salud, consumo de la cola y retiro seguro de workers.
- **Operación y recuperación:** instrucciones de despliegue, verificación, reconstrucción y retiro de recursos, junto con el manejo de configuración y secretos.
- **Capacidad, costo y limitaciones:** configuración efectiva, estimación y consumo observado, puntos únicos de falla restantes y relación con los resultados de las pruebas.

Ubicación: `docs/entrega3/arquitectura.md`, referenciado desde `README.md`. Incluir los archivos fuente de los diagramas y los scripts o instrucciones utilizados.

Verificación funcional del frontend
Documentar los recorridos mínimos de estudiante y profesor definidos en **Frontend mínimo funcional**, con las rutas de interfaz y las evidencias de verificación. El recorrido conectado de autoría, publicación, inscripción, consumo y evaluación deberá ejecutarse sobre el entorno cloud.

Ubicación: directorio `docs/entrega3`, referenciado desde `README.md`.

Análisis de capacidad y pruebas de carga comparativas

Diseño del experimento
Repetir sobre la arquitectura elástica los **dos escenarios de la entrega 2** y añadir un tercer escenario dedicado al autoscaling de ambas capas. Las solicitudes a la API deberán ingresar por el balanceador; las transferencias directas de archivos conservarán su ruta al almacenamiento de objetos.

Usar la misma herramienta y versión de la entrega anterior cuando sea posible. Mantener equivalentes los datos, recorridos, mezcla de operaciones, usuarios virtuales o tasa de llegada, pausas, duración, calentamiento y características de los archivos. Registrar también los recursos del generador, su ubicación, las versiones de software, los tamaños de máquinas, los límites de conexiones y la configuración de workers.

Ejecutar el generador fuera de las máquinas de aplicación y comprobar que no limita las mediciones. Repetir las corridas comparadas para informar su variación. Mantener constantes los parámetros ajenos al cambio estudiado; cualquier diferencia deberá declararse y discutirse como posible causa de los resultados.

Si falta una línea base reproducible, reconstruirla con el commit y la configuración de la entrega 2, identificarla como medición retrospectiva y conservar sus evidencias. Esa ejecución sirve únicamente como referencia: los resultados de la entrega 3 deben corresponder al entorno elástico.

Definir antes de las pruebas los criterios de éxito, degradación y parada. Separar errores de infraestructura, fallos funcionales y rechazos esperados. Un estado HTTP exitoso no reemplaza la validación del resultado de negocio.

Escenario 1 — Actividad académica concurrente
Reproducir la consulta del catálogo, acceso a cursos, inscripción, consulta de contenido, registro válido de progreso y presentación de quizzes. Utilizar cuentas e intentos independientes y conservar la mezcla de lecturas y escrituras de la línea base.

Comparar latencias, throughput y errores, relacionándolos con la carga de las instancias, la cola cuando corresponda y PostgreSQL. Verificar la integridad de intentos, calificaciones y progreso, incluido el envío duplicado. Identificar si el límite se desplaza desde la API hacia la base de datos u otra dependencia compartida.

Escenario 2 — Carga, procesamiento y consumo multimedia
Reproducir las cargas de profesores y el consumo concurrente de contenido HLS. Mantener los perfiles de archivos, rendiciones y patrón de transferencias de la entrega 2. Conservar la concurrencia por instancia de worker cuando sea comparable y registrar cualquier diferencia; la cantidad total de workers variará por autoscaling y constituye parte del cambio arquitectónico evaluado. Registrar tanto las instancias activas como la concurrencia total efectiva durante cada corrida.

Separar las mediciones de autorización y confirmación de carga, transferencia al almacenamiento, espera en cola, procesamiento y tiempo hasta el estado `available`. Registrar trabajos completados, reintentos, fallos, profundidad y antigüedad de la cola, y observar su drenaje al terminar la generación de carga.

Distinguir el tráfico de control atendido por la API de los archivos servidos por el almacenamiento de objetos. Analizar por separado el efecto del escalamiento de la API y de los workers sobre la admisión de cargas, la espera en cola, la tasa de procesamiento y el tiempo hasta disponibilidad del contenido. Las mejoras de descarga o reproducción no deben atribuirse al autoscaling de la API sin evidencia que respalde esa relación.

Escenario 3 — Ciclo de escalamiento automático
Ejecutar dos variantes obligatorias, con configuración y resultados diferenciados: **3A, escalamiento de la API**, y **3B, escalamiento de workers**. Cada una deberá producir un ciclo observable:

1. Iniciar con baja carga y una instancia operativa en el grupo evaluado.
2. Aumentar las solicitudes en 3A o la llegada de trabajos multimedia en 3B hasta provocar la creación automática de al menos una instancia adicional.
3. Sostener la carga durante el arranque y la estabilización; comprobar atención de solicitudes por nuevas réplicas en 3A y procesamiento efectivo por nuevos workers en 3B.
4. Reducir la demanda y observar la reducción automática. En 3B, registrar el drenaje de la cola y el tratamiento seguro de los trabajos activos.
5. Verificar que las operaciones, los datos y los trabajos aceptados conservan su integridad durante el ciclo.

Registrar por grupo una línea de tiempo con demanda, señal de escalamiento, decisión del servicio, instancias creadas y operativas, incorporación al balanceador o inicio de consumo de la cola y retiro. Correlacionar estos eventos con latencias y errores de la API, espera en cola, trabajos completados, reintentos y tiempo hasta disponibilidad del contenido.

Incluir una comprobación controlada de retiro de un worker con un trabajo activo y demostrar su finalización o recuperación segura. Esta comprobación complementa el ciclo automático y no reemplaza la evidencia de reducción por autoscaling. Al terminar, reconciliar los trabajos aceptados con sus estados finales o fallos diagnosticables para detectar pérdidas o efectos duplicados.

Distinguir el tiempo desde el incremento de carga hasta la decisión de escalamiento y el tiempo desde esa decisión hasta que la nueva capacidad atiende solicitudes o procesa trabajos, según el grupo. No basta con mostrar el número final de máquinas. Si la primera corrida no activa el mecanismo, revisar la carga, la señal y la configuración y repetir el experimento dentro del presupuesto.

Métricas e informe

Reportar por escenario y corrida:

- Latencia promedio, p50, p95 y p99 de las operaciones medidas, throughput, errores y timeouts.
- CPU, memoria y, cuando sean relevantes, red y disco de API y workers.
- Conexiones y carga de PostgreSQL; profundidad, antigüedad y tasa de procesamiento de la cola.
- Número de instancias existentes y operativas a lo largo del tiempo, separado para API y workers, y concurrencia efectiva de procesamiento.
- Tiempos de reacción, arranque e incorporación de capacidad para el escenario de autoscaling.

Las métricas de cola y procesamiento deben incluirse en los escenarios que las ejercitan. Identificar unidades, ventanas de agregación y cantidad de observaciones. No mezclar las latencias HTTP con la duración de trabajos asíncronos ni promediar percentiles de corridas como si fueran el percentil del conjunto.

El informe deberá incluir parámetros completos, configuración efectiva, scripts, datos sintéticos, resultados originales, gráficas, limitaciones y una tabla comparativa para los escenarios 1 y 2:

Escenario y operación	Nivel de carga	Métrica y unidad	Entrega 2	Entrega 3	Cambio observado y explicación
Completar con cada comparación	Misma demanda o diferencia declarada	Latencia, throughput, errores u otra métrica	Valor medido	Valor medido	Interpretación sustentada

Responder qué mejoró, qué permaneció igual o empeoró, bajo qué condiciones y por qué. Explicar cuándo se activó el escalamiento, qué ocurrió mientras arrancaban las instancias, qué dependencia limita ahora la capacidad y qué costo adicional implicó el cambio.

No se exige demostrar una mejora en todas las métricas. Se exige una arquitectura que escale automáticamente y un análisis riguroso de sus efectos. No presentar el máximo probado como capacidad máxima si no se alcanzó la saturación, ni extrapolar el resultado a una escala no evaluada.

Ubicación: `capacity-planning/pruebas_de_carga_entrega3.md`, con enlaces a los scripts, datos de prueba, resultados originales y gráficas.

Calidad del código y deuda técnica

Primer análisis y establecimiento de la línea base

Esta entrega constituye el primer ejercicio formal de identificación y documentación de deuda técnica del proyecto. El equipo deberá realizar un diagnóstico del estado actual de la solución y establecer una línea base verificable para dar seguimiento a su evolución en entregas posteriores. No se

exige reconstruir reportes de entregas anteriores, comparar con la entrega 2 ni demostrar una reducción respecto de un valor previo.

Ejecutar SonarQube sobre el código de la entrega mediante el pipeline de integración continua. Se admite una herramienta equivalente si el equipo documenta la correspondencia de indicadores, severidades y evidencias. Para GitHub se utilizará GitHub Actions; para GitLab, su mecanismo de integración continua equivalente.

El análisis automatizado es obligatorio. La automatización existente deberá mantenerse operativa y conservar la ejecución de las pruebas que respaldan las correcciones o refactorizaciones realizadas. No se exige automatizar el despliegue cloud.

Indicador	Requisito de la entrega 3
Duplicación de código	Máximo 20%
Mantenibilidad	Calificación C o mejor: A, B o C
Deuda técnica total estimada por la herramienta	Registrar el valor, su unidad y su interpretación como primera medición de referencia

Estos indicadores se medirán sobre el código total analizado del proyecto, incluidos API, workers y frontend, no únicamente sobre código nuevo. Documentar la fecha del análisis, el commit evaluado, la herramienta y su versión, el perfil de reglas, el alcance y las exclusiones justificadas. No excluir código de aplicación ni alterar artificialmente el alcance para aparentar cumplimiento de los umbrales.

El reporte correspondiente al **commit final de la entrega 3** y el inventario de pendientes asociados constituirán la línea base para las siguientes entregas. Si se realizan análisis adicionales durante el sprint, podrán utilizarse para orientar y verificar correcciones, pero no se exige una comparación histórica ni un porcentaje de reducción en este primer ejercicio.

La estimación de deuda técnica de la herramienta deberá interpretarse junto con los hallazgos y sus impactos. No sustituye la revisión de decisiones arquitectónicas, configuración, pruebas y operación, cuyas limitaciones también deberán registrarse cuando representen deuda técnica.

Primer documento de deuda técnica

Crear el documento `docs/entrega3/deuda-tecnica.md` a partir del diagnóstico del código y de la solución efectivamente desplegada. Su propósito es identificar los compromisos técnicos presentes, explicar sus consecuencias y definir cómo se atenderán. No se requiere clasificarlos como deuda heredada, persistente o nueva frente a entregas anteriores.

El documento deberá incluir:

- **Alcance y método del diagnóstico:** componentes revisados, herramientas utilizadas y actividades de revisión realizadas por el equipo.
- **Resumen de la medición inicial:** duplicación, mantenibilidad y deuda técnica estimada, con unidades, interpretación y enlace al reporte del commit final.
- **Inventario de deuda identificada:** hallazgos de código, arquitectura, infraestructura, pruebas u operación, sustentados en evidencia concreta.

- **Priorización y plan de tratamiento:** orden de atención, acciones previstas, responsables y plazos o entrega objetivo.
- **Correcciones realizadas durante este ejercicio, si las hubo:** descripción, commit y evidencia de verificación; distinguirlas de los pendientes que integran la línea base final.

Para cada elemento del inventario, registrar como mínimo:

Campo	Contenido esperado
Identificador y categoría	Código único y tipo de deuda
Descripción y ubicación	Problema concreto y componente, archivo o configuración afectados
Evidencia	Regla del analizador, resultado de prueba, métrica o decisión documentada que respalda el hallazgo
Impacto	Consecuencia sobre mantenibilidad, seguridad, desempeño, confiabilidad o costo
Prioridad y justificación	Orden de atención según impacto, urgencia y dependencias
Tratamiento propuesto	Corrección o mitigación y criterio para verificar su resolución
Responsable y plazo	Integrante responsable y fecha o entrega prevista para atenderlo
Estado y decisión	Pendiente, en tratamiento o resuelto; justificar el aplazamiento o la aceptación temporal cuando corresponda

No basta con copiar la lista de SonarQube. El equipo deberá interpretar los hallazgos y relacionarlos con el funcionamiento de la solución. Una funcionalidad futura fuera del alcance no constituye por sí sola deuda técnica; documentar el compromiso técnico y sus consecuencias. Los incumplimientos de requisitos obligatorios de esta entrega no se subsanan únicamente registrándolos como deuda.

Todo hallazgo **Major o superior** que permanezca abierto deberá quedar registrado con su identificador o regla, ubicación, descripción, impacto y motivo de aplazamiento. Los hallazgos **Critical o Blocker** deberán resolverse antes del cierre o contar con un plan explícito que señale pasos, responsable y plazo. Si la herramienta utiliza otra clasificación, documentar la equivalencia aplicada.

Entregar el reporte completo o una exportación verificable asociado al commit final de la entrega 3, junto con el enlace a la ejecución del pipeline. Las capturas parciales y las explicaciones verbales no sustituyen el reporte. Se evaluará la calidad del diagnóstico, la trazabilidad de la línea base y la pertinencia de la priorización y del plan de tratamiento.

Entregables finales

1. **Plataforma MOOC desplegada en la nube pública**, con frontend mínimo funcional, autoscaling en API y workers, cola compartida y persistencia operativa.

2. **Release del código y la configuración**, incluidos código Go y frontend, contenedores, plantillas, mecanismos de arranque, políticas de escalamiento y configuración de balanceo.
3. **Documento de arquitectura** en `docs/entrega3/arquitectura.md`, con instrucciones de operación y reconstrucción.
4. **Evidencia de verificación funcional del frontend**, con la matriz de recorridos y rutas de estudiante y profesor en `docs/entrega3`.
5. **Informe de capacidad** en `capacity-planning/pruebas_de_carga_entrega3.md`, con los tres escenarios, las variantes 3A y 3B y la comparación de los dos escenarios heredados.
6. **Primer documento de deuda técnica** en `docs/entrega3/deuda-tecnica.md`, con diagnóstico, línea base del commit final, inventario priorizado y plan de tratamiento.
7. **Evidencias reproducibles**, incluidos scripts, datos sintéticos, resultados originales, reportes de calidad y ejecuciones del pipeline, enlazados desde los informes.
8. **Video de sustentación**, enlazado desde el README y accesible para el equipo docente.

Toda la entrega se realizará mediante el repositorio del grupo en **GitHub o GitLab**, de acuerdo con el programa del curso. Crear un tag o release identificable, por ejemplo `entrega-3`, y registrar el commit evaluado. El README deberá ofrecer acceso a la documentación, las instrucciones de ejecución, las URL HTTPS del frontend y del balanceador de API y las evidencias, sin publicar credenciales. El procedimiento de acceso para el equipo docente se comunicará por el medio privado dispuesto en el curso.

Sustentación

Video

Cada grupo preparará un video que presente la arquitectura desplegada, los recorridos mínimos de estudiante y profesor desde el frontend, los ciclos de escalamiento de API y workers con sus evidencias, los resultados comparativos de desempeño y el primer diagnóstico de calidad y deuda técnica, con su línea base y plan de tratamiento.

Demostrar el recorrido conectado profesor–estudiante definido en **Frontend mínimo funcional**, tráfico distribuido entre réplicas de API, incorporación y retiro automáticos de capacidad en ambos grupos, procesamiento por nuevos workers y recuperación segura de trabajos. Verificar operaciones sobre PostgreSQL administrado, acceso al almacenamiento de objetos y continuidad funcional durante los cambios de capacidad. Los procesos largos podrán mostrarse mediante ejecuciones previas, siempre que sus registros completos estén disponibles.

Sustentación síncrona

El equipo docente podrá solicitar una sustentación síncrona o repetir una prueba. Si los recursos se retiraron por control de costos, el equipo deberá recrearlos previamente con la configuración documentada. Todos los integrantes deberán poder explicar el trabajo realizado.

La sustentación es obligatoria. No presentar la sustentación equivale a no realizar la entrega.